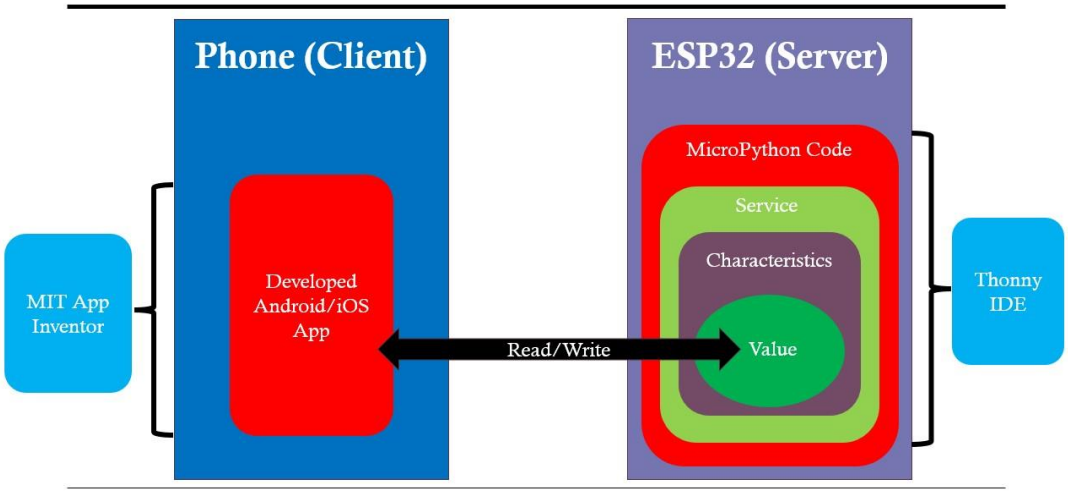
Controlling a NeoPixel

via a Bluetooth App

A Step-by-Step Project Guide

ESP32 | MicroPython | MIT App Inventor | NeoPixel LED Ring



Technologies Used

ESP32 Microcontroller	MicroPython (Thonny IDE)	MIT App Inventor	WS2812B NeoPixel

1. Project Overview

1.1 What This Project Does

This project demonstrates wireless communication between a smartphone and an ESP32 microcontroller using Bluetooth Low Energy (BLE). A custom Android/iOS app built using MIT App Inventor sends simple text commands over BLE to the ESP32. The ESP32 runs MicroPython code that listens for those commands and responds by triggering different LED animations on a 16-pixel NeoPixel ring.

In short: you tap a button on your phone, and the lights on the NeoPixel ring change — all wirelessly, without any Wi-Fi or internet connection needed.

1.2 How Bluetooth Low Energy (BLE) Works in This Project

The diagram below captures the architecture of this project. Let us walk through each part:

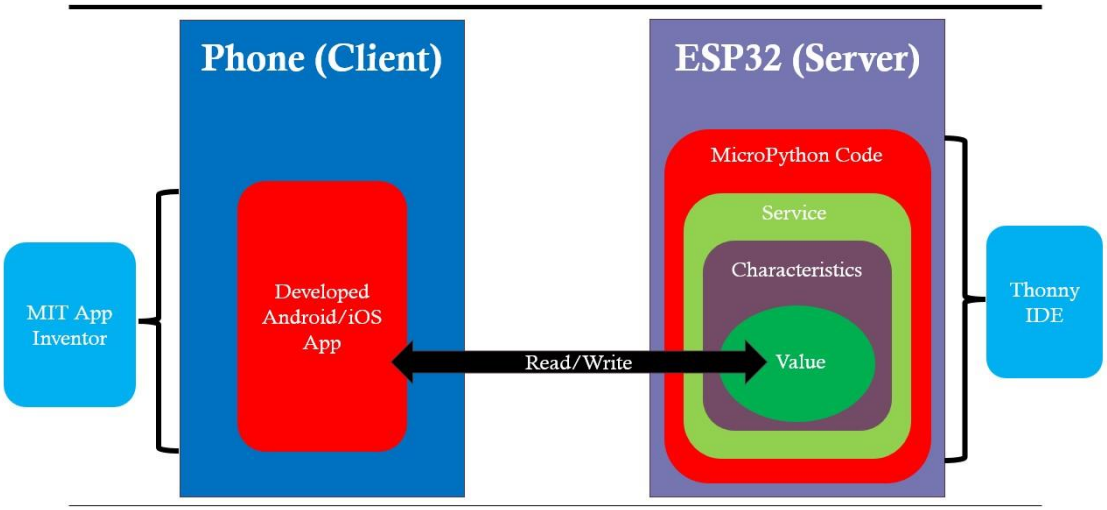


Figure 1: BLE Communication Architecture — Phone (Client) and ESP32 (Server)

The ESP32 — The Server

The ESP32 acts as the BLE Server. It advertises its presence so that nearby devices can discover and connect to it. Inside the server, there is a layered structure:

- Service: Think of this as a category or a bag. The ESP32 exposes one BLE Service identified by a unique UUID. This groups all related data together.
- Characteristic: Inside the Service sits a Characteristic — a specific data slot like a file inside the bag that can hold a value. In this project, it is set up with WRITE permission, meaning the phone can write data into it.
- Value: The actual data, in this case a simple text string like "1" or "2" is stored in the Value field of the characteristic.

The MicroPython code on the ESP32, written using **Thonny IDE**, handles all of this: setting up the BLE service, listening for incoming data, and triggering the correct NeoPixel animation.

The Phone — The Client

The smartphone acts as the BLE Client. It scans for nearby BLE devices, connects to the ESP32 (identified by its name, e.g. "ESP32-Nayan"), and then writes command values (like "1" or "2") to the ESP32's BLE characteristic.

The phone app is built using **MIT App Inventor** — a visual block-based programming platform that makes it easy to create Android/iOS apps without traditional coding.

The Read/Write Arrow

The arrow in the diagram labeled "Read/Write" shows the bidirectional data channel between the phone and the ESP32. In this project we primarily use Write — the phone writes a command to the ESP32's characteristic. The ESP32 then reads that value and acts on it.

Key Insight

BLE is different from classic Bluetooth. It is designed for very low power consumption and short bursts of data — perfect for microcontroller projects like this one. The phone and ESP32 don't stay continuously streaming; instead, the phone sends a short command only when you press a button.

2. Step 1 — Building the Hardware (ESP32 + NeoPixel)

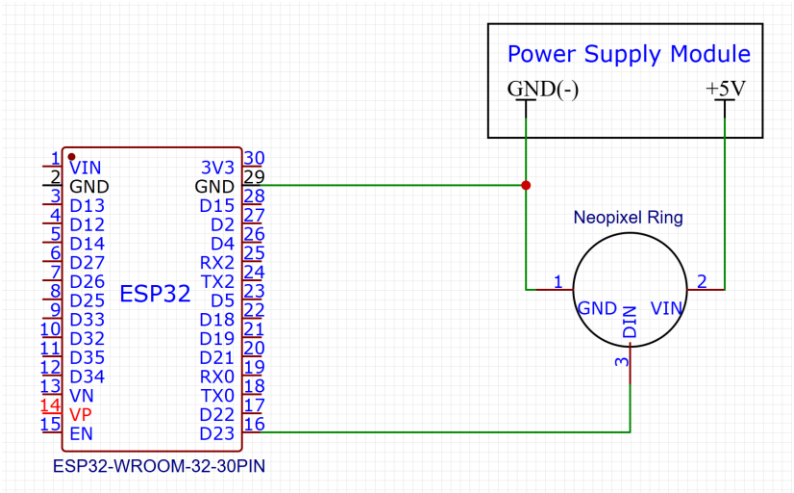
2.1 Components You Need

Component
ESP32 Development Board
NeoPixel Ring (WS2812B, 16 pixels — connected to GPIO Pin 22)
Jumper Wires
USB Cable (for programming via Thonny IDE)
5V Power Supply (dedicated 5 V adapter to power the NeoPixel ring)
Breadboard (for prototyping all connections before any permanent wiring)

2.2 Wiring the NeoPixel to the ESP32

A NeoPixel (WS2812B) has three connections. Wire them as follows:

NeoPixel Pin	ESP32 Pin	Notes
5V / VCC	`5V`	Power supply for the NeoPixel LEDs.
GND	`GND`	Common ground — must be shared between ESP32 and NeoPixel.
DIN (Data In)	`GPIO 22 or 23`	Data pin. This is defined in the code as Pin(22).



Tip — Modifying the Data Pin

If you want to use a different GPIO pin (e.g., GPIO 5), you only need to change Pin(22) to Pin(5) in the MicroPython code. See Section 3 for details.

Maker's Hack — Powering NeoPixel from ESP32's Vin Pin

In a pinch, you can connect the NeoPixel's 5V/VCC pin directly to the Vin pin on the ESP32. Vin passes through the USB power (usually 5 V) when the ESP32 is connected via USB. For a 16-pixel ring at moderate brightness this is often enough. CAUTION: Do not do this if you plan to run all LEDs at full white brightness simultaneously — a 16-pixel ring can draw up to 960 mA at full white, which exceeds what most USB ports and the ESP32's on-board regulator can safely supply. Excessive current draw can damage the ESP32 or cause unstable behaviour. Always keep brightness values below 150 (out of 255) when using Vin as power.

2.3 Test Your Hardware with a Dummy Code

Before writing the full BLE project code, always test your hardware first. A simple dummy test confirms that your NeoPixel is correctly wired, the data pin is correct, and the LEDs respond as expected. Run this test code in Thonny before moving on.

3. Step 2 — Writing the Server Code (MicroPython in Thonny)

3.1 Complete Code

The complete is available in GitHub Repository under Bluetooth Project file. Code name : BT_Code_ControllingNeoPixel.

Below is each section explained in detail.

3.1.1 Importing Necessary libraries and Initialization :

```
from machine import Pin
import bluetooth
import time
import neopixel
import random

my_neo = neopixel.NeoPixel(Pin(22), 16)
value = ""
name = "ESP32-Nayan"   # Change this to your device name
ble = bluetooth.BLE()

ble.active(False)
time.sleep(0.5)
ble.active(True)
ble.config(gap_name=name)
```

- **my_neo = neopixel.NeoPixel(Pin(22), 16)** — This creates the NeoPixel object. The first argument, Pin(22), tells MicroPython which GPIO pin the NeoPixel data line is connected to. The second argument, 16, is the total number of LEDs in your ring or strip. Internally, MicroPython uses this to allocate a memory buffer of the right size (3 bytes per LED: R, G, B).
- **value = " "**— This declares a global variable named value and sets it to an empty string. Its job is simple: hold the most recent command received from the phone over BLE. It starts empty because no command has been received yet. The main while loop checks this variable continuously.
- **name = "ESP32-Nayan"**— This is the BLE device name that will appear on your phone when scanning for Bluetooth devices. You should change this to something unique — especially useful in a classroom where multiple ESP32 boards may be running the same project simultaneously.
- **ble = bluetooth.BLE()**— This creates the BLE (Bluetooth Low Energy) object. Think of it as the main controller for all BLE operations — advertising, connecting, reading, and writing. All BLE actions in this program go through this object.
- **ble.active(False) followed by ble.active(True)**— Resetting the BLE radio before starting is a good practice. If the ESP32 was previously connected or in an unexpected state (for example, after a reboot), this ensures the BLE hardware starts cleanly. The 0.5-second sleep gives the hardware time to fully power down before restarting.

Modifiable: Number of Pixels

Change the 16 in NeoPixel(Pin(22), 16) to match however many LEDs your ring or strip has. For example, a 12-pixel ring: NeoPixel(Pin(22), 12). Also update the range(0, 16) in both animation functions accordingly.

3.1.2 BLE Service and Characteristic Registration

```
service_UUID = bluetooth.UUID("6e400001-b5a3-f393-e0a9-e50e24dcca9e")
char_UUID    = bluetooth.UUID("6e400002-b5a3-f393-e0a9-e50e24dcca9e")

char = (char_UUID, bluetooth.FLAG_WRITE)
service = (service_UUID, (char,),)
((char_handle,),) = ble.gatts_register_services((service,))
```

This section defines the BLE GATT (Generic Attribute Profile) structure — the formal definition of what data the ESP32 exposes over BLE. Think of it like setting up a bag: the Service is the bag itself, the Characteristic is a file inside the bag where data can be written, and the Value is the actual piece of paper placed in that file.

- **service_UUID**— A 128-bit UUID (Universally Unique Identifier) that labels the BLE Service. UUIDs are used in BLE to identify services and characteristics so that a phone knows what it is connecting to. This particular UUID is based on the widely-used Nordic UART Service — a standard that many BLE apps already recognise.
- **char_UUID**— The UUID for the specific Characteristic within the Service. A Service can contain multiple Characteristics — in this project we only need one. The phone app must target this exact UUID when it writes data; otherwise the ESP32 will not receive the command.
- **bluetooth.FLAG_WRITE**— This flag sets the permission on the characteristic. FLAG_WRITE means the phone (client) is allowed to write data into this characteristic. The ESP32 does not need to “poll” for new data — instead, the BLE hardware automatically fires an interrupt (see Section 3.5) whenever new data arrives.
- **ble.gatts_register_services((service,))**— This registers the service and characteristic with the BLE stack. The function returns a handle — a small integer (stored as char_handle) that acts as an internal reference number. Whenever the code needs to read the incoming data, it uses this handle to look up the right characteristic.

3.1.3 The BLE Event Handler — Receiving Commands

```
def event_occured(event, data):
    if event == 1:
        print("Connected")
    elif event == 2:
        print("Disconnected")
        advertise(name)
    elif event == 3:
        conn_handle, value_handle = data
        if value_handle == char_handle:
            raw_msg = ble.gatts_read(char_handle).rstrip(b'\x00')
            msg = raw_msg.decode().strip()
            print("Received:", msg)
            global value
            value = msg
```

This function is the heart of the BLE communication. It is registered as a callback (an interrupt handler) using ble.irq(event_occured) — this means MicroPython calls this function automatically every time a BLE event occurs, even in the middle of the main loop.

- **event == 1 (Connected)**— Fired when a phone successfully establishes a BLE connection to the ESP32. At this point, the phone and ESP32 have completed the BLE handshake. The code

simply prints a message to Thonny's Shell so you can confirm the connection visually during development.

- **event == 2 (Disconnected)**— Fired when the phone disconnects — either deliberately (by closing the app) or due to signal loss. The key action here is calling advertise(name) again so that the ESP32 immediately begins broadcasting its presence again and can accept the next connection without needing a restart.
- **event == 3 (Write received)**— This is the most important event. It fires when the phone has written data to the BLE characteristic. The code unpacks the data tuple to get conn_handle (the connection ID) and value_handle (which characteristic was written). It checks that the write was to our specific char_handle before reading.
- **ble.gatts_read(char_handle)**— Reads the raw bytes from the characteristic. BLE data arrives as bytes, not as a Python string. The .rstrip(b'\x00') removes any trailing null bytes that BLE sometimes appends to pad the data. The .decode() then converts the bytes to a Python string, and .strip() removes any accidental whitespace.
- **global value**— Python requires the global keyword inside a function to modify a variable that was defined at the module (top) level. Without it, value = msg would create a new local variable inside the function and disappear when the function ends — the main loop would never see the update. The global keyword tells Python to update the shared variable instead.

Why global value?
Python requires the global keyword to modify a variable that was defined outside the current function. Without it, value = msg would create a local variable inside the function, and the main while loop would never see the update.

3.1.4 The Advertise Function

```
def advertise(device_name):
    name_bytes = device_name.encode()
    flags      = bytearray([0x02, 0x01, 0x06])
    short_name = bytearray([len(name_bytes) + 1, 0x08]) + name_bytes
    full_name  = bytearray([len(name_bytes) + 1, 0x09]) + name_bytes
    adv_data   = flags + short_name + full_name
    ble.gap_advertise(50, adv_data=adv_data)
    print("Advertising as:", device_name)
```

This function manually builds the raw advertisement packet that the ESP32 broadcasts over the air — the bytes that nearby phones pick up when scanning for BLE devices. Understanding this function is not required to use the project, but it explains why the ESP32 shows up with the correct name on your phone.

- **device_name.encode()**— Converts the Python string (e.g., "ESP32-Nayan") into raw bytes. BLE advertisement packets carry raw bytes, not Python strings, so this conversion is essential.
- **flags = bytearray([0x02, 0x01, 0x06])**— These three bytes are the standard BLE advertisement flags. 0x02 is the length (2 bytes follow), 0x01 is the data type code for flags, and 0x06 means "General Discoverable Mode, BR/EDR not supported" — in plain English: "This is a BLE-only device that anyone can discover."
- **0x08 (Shortened Local Name) and 0x09 (Complete Local Name)**— These are BLE AD type codes. Including both the short name and the full name in the advertisement improves compatibility across different phones, operating systems, and BLE scanner apps. Some phones show the short name first during scanning; others prefer the full name.

- **ble.gap_advertise(50, adv_data=adv_data)**— Starts the BLE advertisement. The first argument, 50, is the advertisement interval in milliseconds. A smaller interval means faster discovery by the phone but slightly more power consumption. For classroom use, 50 ms is a good balance between speed and battery life.

3.1.5 NeoPixel Animation Functions

Animation 1: Star (Solid Blue)

```
def my_neo_star():
    for n in range(0, 16):
        my_neo[n] = (0, 0, 150)
    my_neo.write()
```

Sets all 16 LEDs to the same colour: Red=0, Green=0, Blue=150 : a medium blue. The loop assigns each LED its colour value, and my_neo.write() sends the data out to the LEDs all at once.

Colour format: `(R, G, B)` where each value is 0–255. Examples: (255,0,0) = red, (0,255,0) = green, (255,255,255) = white, (0,0,0) = off.

Animation 2: Bling (Random Pastel Colours)

```
def my_neo_bling():
    for k in range(0, 16):
        r = random.randint(100, 150)
        g = random.randint(100, 150)
        b = random.randint(100, 150)
        my_neo[k] = (r, g, b)
        time.sleep(0.1)
        my_neo.write()
```

Lights up each LED one by one with a random pastel colour. The range 100–150 keeps all colours in a soft, medium-brightness range (avoiding very dark or oversaturated colours). The 0.1 second delay creates a chasing/twinkling effect.

3.1.6 The Main Loop — Responding to Commands

```
while True:
    if value == "1":
        print("NeoPixel_Star")
        my_neo_star()
        value = ""
    elif value == "2":
        print("NeoPixel_Bling")
        my_neo_bling()
        value = ""
    time.sleep(0.2)
```

This is the continuously running loop that is the brain of the program. It runs forever (while True:) and checks the value variable every 0.2 seconds. Recall that value is updated by the event_occured interrupt handler whenever the phone writes a BLE command.

- **if value == "1"**— Checks whether the last received BLE command was the string "1". If so, it calls the my_neo_star() function, which sets all 16 LEDs to a solid blue colour. This command is sent by pressing the first button in the phone app.
- **elif value == "2"**— Checks for the command "2". If received, it calls my_neo_bling(), which lights each LED with a random pastel colour in sequence. This is sent by the second button in the phone app.
- **value = " " (resetting after action)**— After running the animation, the value is reset to an empty string. This is important: without this reset, the while loop would see "1" or "2" on every cycle and run the animation repeatedly, even if you only pressed the button once. Clearing the value ensures each button press triggers exactly one animation.
- **time.sleep(0.2)**— Adds a 200 ms pause at the end of each loop cycle. This prevents the loop from consuming all the processor's time and potentially interfering with the BLE stack, which also needs CPU time to handle incoming packets. It also means the maximum response time to a button press is about 200 ms — fast enough to feel instant.

Extending the Project — Adding More Commands

To add more animations, simply add more elif blocks in the while loop. For example: elif value == "3": my_neo_rainbow() value = "" Then create the corresponding def my_neo_rainbow(): function, and add a new button in the MIT App Inventor app that sends "3".

3.9 Quick Reference — What to Modify

What to Change	Where in the Code	Example
Device BLE name	`name = "ESP32-Nayan"`	`name = "ESP32-Lab1"`
Data pin number	`neopixel.NeoPixel(Pin(22), 16)`	`Pin(5) for GPIO5`
Number of LEDs	`NeoPixel(Pin(22), 16)`	`Change 16 to 12 or 24`
LED colour (Star)	`my_neo[n] = (0, 0, 150)`	`(255, 0, 0) for red`
Random colour range	`random.randint(100, 150)`	`randint(50, 255) for vivid`
Animation speed	`time.sleep(0.1) in bling`	`0.05 for faster`
Add a new animation	`New elif in while loop`	`elif value == "3": ...`

4. Step 3 — Building the Client App (MIT App Inventor)

4.1 What is MIT App Inventor?

MIT App Inventor (appinventor.mit.edu) is a free, browser-based visual programming environment developed by MIT. It uses a drag-and-drop block interface — similar to Scratch — to create fully functional Android and iOS apps without needing to write Java or Swift code.

For this project, we will build an app that:

- Scans for and connects to the ESP32 over BLE.
- Displays two buttons — one for each NeoPixel animation.
- Sends the command string ("1" or "2") to the ESP32 when a button is pressed.

4.2 Setting Up the Project

1. Open your browser and go to appinventor.mit.edu. Sign in with a Google account.
2. Click "Start new project" and name it NeoPixelBLE.
3. You are now in the Designer view, which has two panels: the Component panel (left/centre) and the Properties panel (right).

4.3 Adding the BluetoothLE Extension

MIT App Inventor does not include BLE support by default. You need to import an extension:

4. Download the BluetoothLE extension (.aix file) from MIT's official App Inventor extensions page. (Shared in the LMS as well)
5. In App Inventor, scroll to the bottom of the left panel and click "Extension".
6. Click "Import extension" and upload the .aix file. A BluetoothLE1 component will appear in your component list.

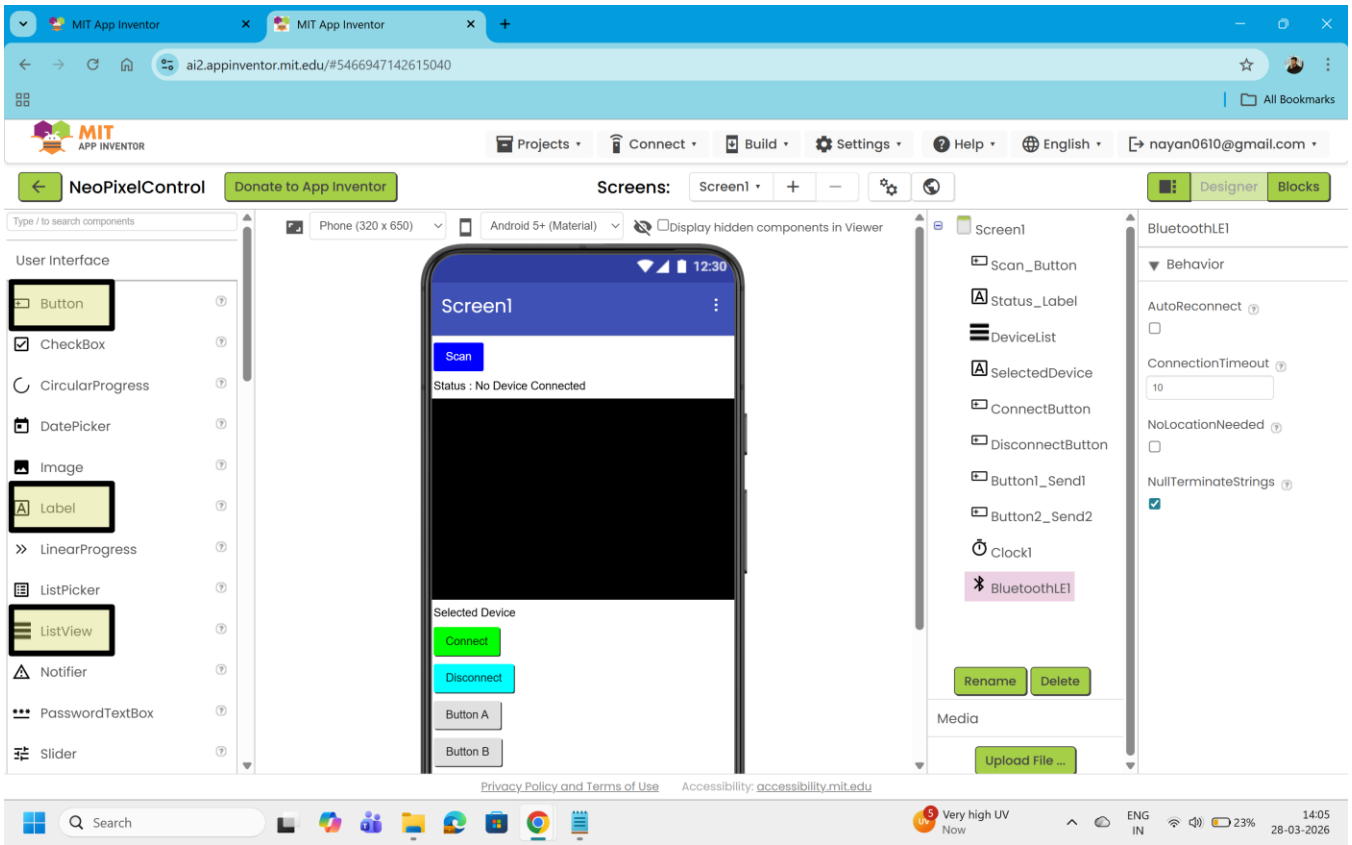
> **Important**
> The BluetoothLE extension is different from the built-in BluetoothClient (which is for Classic Bluetooth). Make sure to use the BluetoothLE extension that matches BLE / GATT — this is what the ESP32 uses.

4.4 Designing the User Interface (Designer View)

In the Designer, add the following UI components by dragging them from the palette:

Component Type	Suggested Name	Purpose
Button	`Scan_Button`	Scan for nearby BLE devices
Label	`Status_label`	Shows system status (Scanning, Connected etc.)
ListView	`DeviceList`	Let the user select the ESP32 from discovered devices
Button	`Connect_Button`	Connect to the selected device
Label	`Selected_Device`	Shows name of the Device selected from ListView
Button	`Button_Send1`	Send command "1" — Star animation
Button	`Button_Send2`	Send command "2" — Bling animation
BluetoothLE1	`(non-visible)`	Handles all BLE operations (from extension)
Clock	`(non-visible)`	For scanning up to 3 seconds

Note : Clock Component is available under Sensors Section in the left Panel.



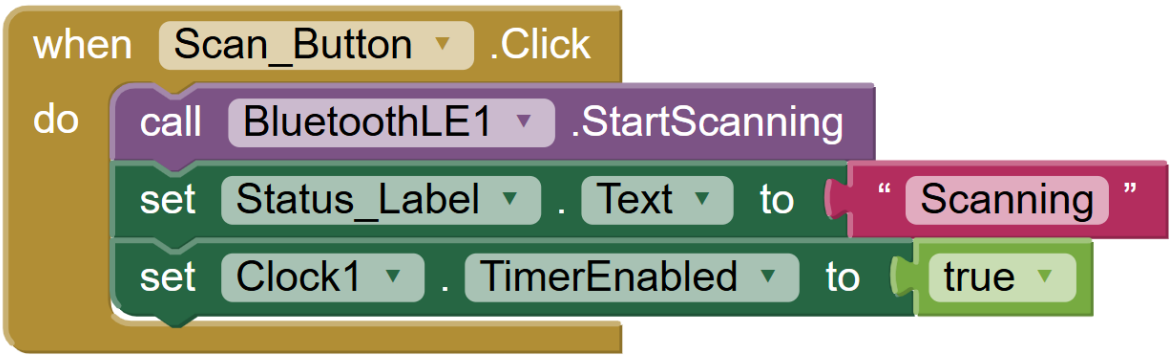
4.5 Programming the Blocks (Blocks View)

Switch to Blocks view (top-right toggle). Here you will define the logic of the app using visual blocks.

Block 1: Scan for BLE Devices

When Scan_Button is clicked :

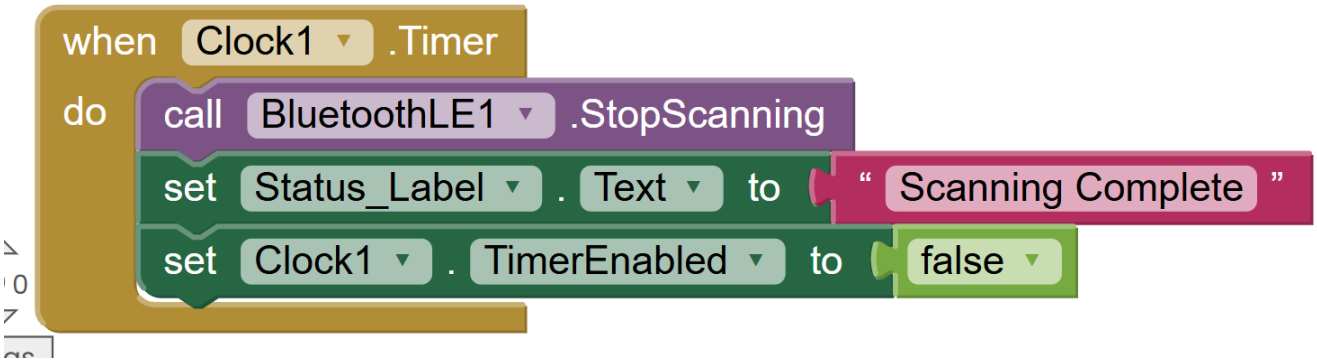
1. Start scanning,
2. Change status label to “Scanning”
3. Enable / Start the timer



Block 2: Stop Scan after 3 seconds (you may change the time in designer block of clock)

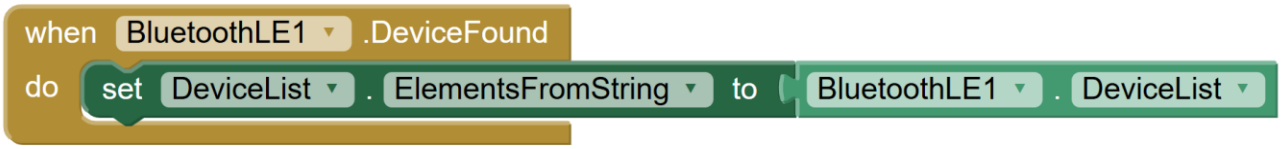
When the set time of the timer is elapsed :

1. Stop Scanning
2. Change status label to “Scanning Complete”
3. Disable /Stop the time



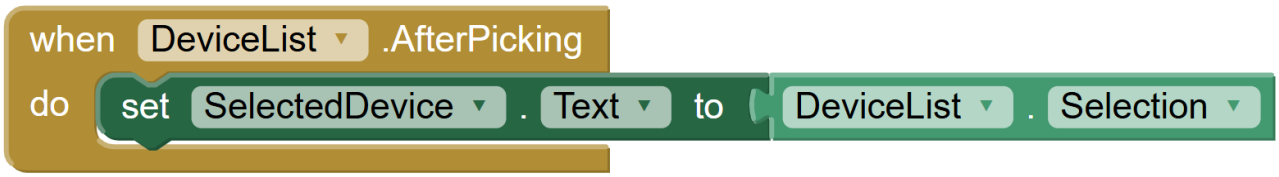
Block 3: Populate the DeviceList

When a Bluetooth Device is found, add it to the Device List



Block 4: Display the name of the Selected Device

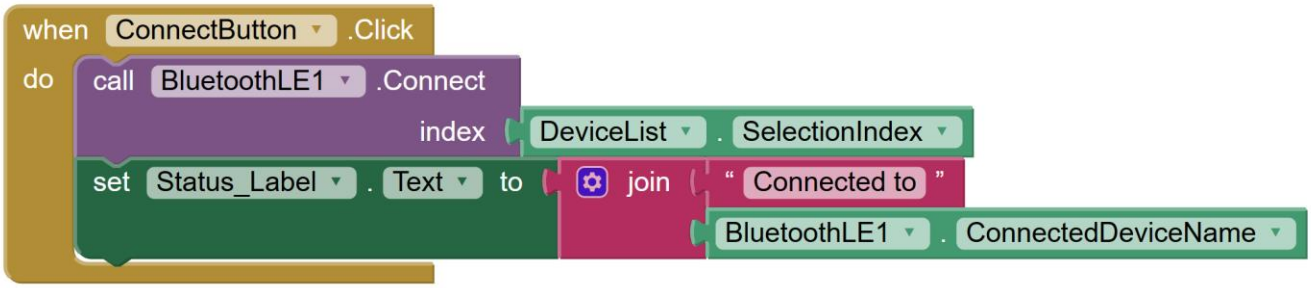
After the user picks a device from the list, the device name is displayed



Block 5: Connect

When the connect_button is pressed :

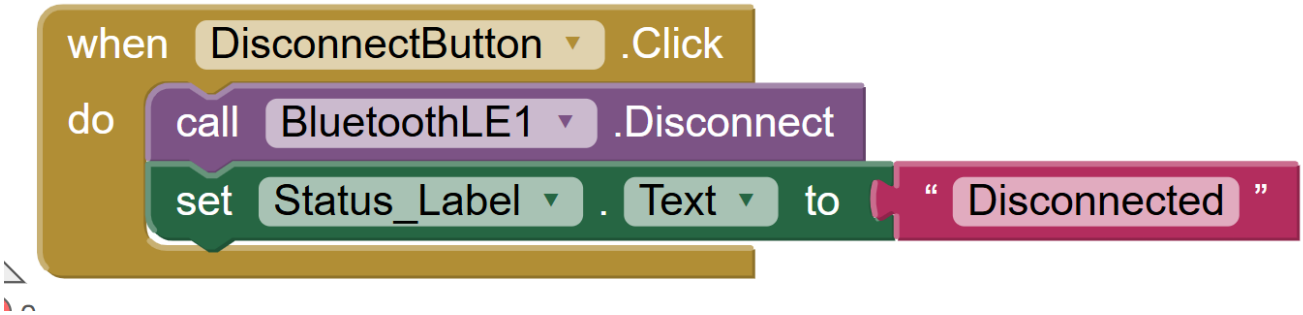
1. Connect to the selcted device.
2. Update the status to : “Connected to “Deivice Name”



Block 6: Disconnect

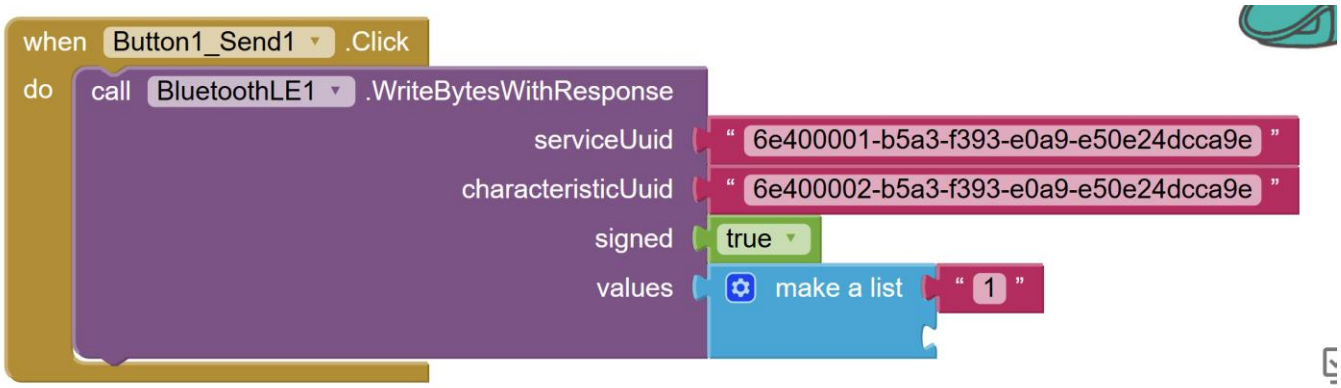
When the Disconnect_Button is pressed :

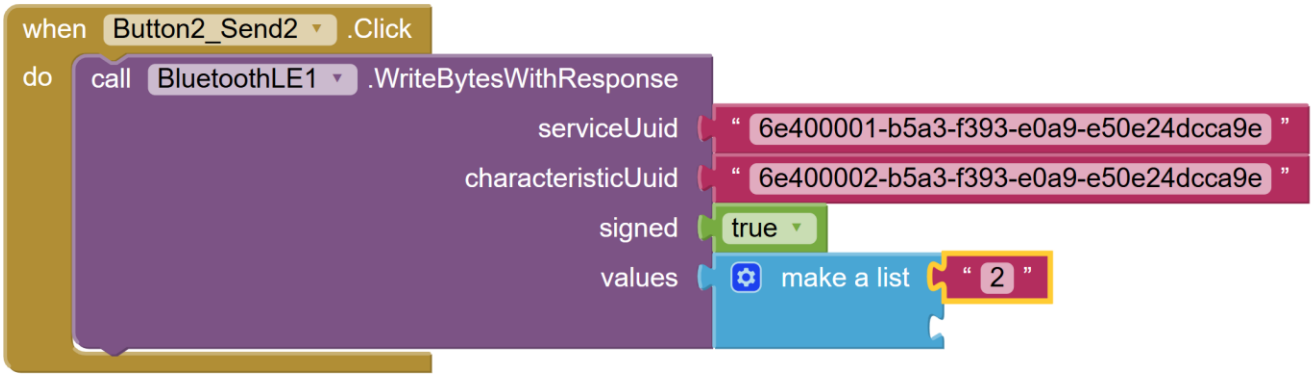
1. Disconnect from the device.
2. Change the status label to “Disconnected”



Block 7 & 8: Send Commands on Button Press

This is the key part. Use `WriteByteWithResponse` to write a text command to the ESP32's characteristic. You must supply the Service UUID and Characteristic UUID — these must exactly match the ones in your MicroPython code.





4.6 Testing the App

7. On your Android phone, install the MIT AI2 Companion app from the Play Store.
8. In App Inventor, go to Connect > AI Companion, and scan the QR code.
9. Make sure the ESP32 is running the MicroPython code (visible in Thonny's Shell as "Advertising as: *whichever name you gave the ESP32*").
10. Press Scan in your app, then select the ESP32 from the list, and press Connect.
11. Press the Star or Bling buttons — the NeoPixel should respond immediately.

Build APK for Permanent Use

Once the app is working, you can export a standalone APK: Build > App (provide QR code for .apk). This lets you install the app permanently on your phone without needing the AI2 Companion.

4.7 User Flow — How a User Uses the App

The user flow describes every tap and action a user takes from opening the app to triggering a NeoPixel animation. Understanding this sequence helps you test the app methodically and explain it to others.

Step-by-Step User Flow

1. Power on the ESP32 (connect via USB or 5V supply). The ESP32 starts advertising over BLE automatically. Open Thonny to confirm you see "Advertising as: *DeviceName*" in the Shell.

2. Open the App on your Android phone (either via the AI2 Companion during development, or the installed APK).

3. Press the Scan button. The app instructs the BluetoothLE1 component to start scanning. After a moment, nearby BLE devices are discovered.

4. Select Device from the List (ListView). The app populates the list with discovered device names and addresses. Scroll to find your ESP32 (e.g., ESP32-Nayan) and tap it.

5. Press the Connect button. The app calls BluetoothLE1.ConnectToDeviceWithAddress using the address selected. The status label updates to "Connected".

6. Press the Button A. The app writes the string "1" to the ESP32's BLE characteristic. The ESP32 receives it, and all LEDs light up in a solid blue pattern.

7. Press the Button B. The app writes "2". The LEDs cycle through random pastel colours.

8. Press the Disconnect button. The ESP32 detects the disconnection and immediately begins advertising again, ready for the next connection.

5. Project Summary & Extensions

5.1 What You Have Built

Congratulations! You have built a complete end-to-end IoT project! Here is a summary of the full system:

Layer	Technology	Role
Server Hardware	ESP32 + WS2812B NeoPixel (16 LEDs)	Physical device that displays LED animations
Server Software	MicroPython on ESP32 via Thonny IDE	BLE server, receives commands, drives NeoPixel LEDs
Wireless Protocol	Bluetooth Low Energy (BLE / GATT)	Wireless link between phone and ESP32
Client Software	Android/iOS App via MIT App Inventor	User interface to send LED commands

5.2 Ideas for Extending the Project

- Add more animations (rainbow, chase, pulse) by adding new functions and elif blocks.
- Add a colour picker in the app to let users choose custom colours and send RGB values as a comma-separated string like "R,G,B".
- Add a slider in the app to control brightness and send the value to the ESP32.
- Control multiple NeoPixel strips by wiring them to different GPIO pins and responding to different command strings..

Happy Making!